

MSSV: 23127195

Họ và tên : Trần Mạnh Hùng

CÂU 1:

1.1 Nêu giả định và Phân tích ràng buộc

- **Giả định:** Trong 3 phút đầu tiên, hành vi chủ yếu của 12.600 sinh viên (70% của 18.000) là **Read-heavy** (tải lại trang liên tục để xem lịch và số lượng ghế workshop). Sau đó, traffic chuyển sang **Write-heavy** khi họ bắt đầu tranh nhau bấm đăng ký và thanh toán.
- **Phân tích ràng buộc nghiệp vụ:**
 - *Ngân sách hạn chế (chỉ đủ HA, không đủ FT):* Không thể thiết kế các cụm server đa khu vực (Multi-Region) quá đắt đỏ. Cần tận dụng tối đa tài nguyên hiện có bằng cách phân tải hợp lý (scaling).
 - *Yêu cầu cô lập lỗi:* Ràng buộc "cổng thanh toán sập thì vẫn xem được lịch" bắt buộc thiết kế phải có sự tách biệt (Decoupling) và giới hạn ảnh hưởng (Bulkhead).
 - *Ràng buộc dữ liệu:* "Không bán trùng chỗ" đòi hỏi cơ chế lock hoặc xử lý transaction cực kỳ chặt chẽ tại DB hoặc Cache, không thể dùng Eventual Consistency cho luồng trừ ghế.

1.2. Đề xuất phương án: Phân tích kiến trúc hiện tại và 5 Rủi ro (SPOF)

Dưới đây là 5 rủi ro chết người từ kiến trúc All-in-one hiện tại:

Rủi ro 1: 1 Web/App Server duy nhất (Nút thắt cổ chai tính toán & SPOF)

- **Nó hỏng thế nào:** Lượng truy cập 12.600 request trong 3 phút đầu tạo ra hàng vạn TCP connections và luồng xử lý (threads). Server sẽ cạn kiệt RAM (OOM - Out of Memory) hoặc CPU đạt 100%, dẫn đến treo máy.
- **Người dùng bị ảnh hưởng ra sao:** Toàn bộ hệ thống sập. Sinh viên nhận lỗi 502 Bad Gateway hoặc không thể tải trang.
- **Cải tiến (Chọn gì - Vì sao - Hậu quả nếu sai):**
 - *Chọn gì:* Scale out thành một cụm nhiều App Servers đặt sau 1 **Load Balancer (LB)**.

- *Vì sao:* Chịu được tải đột biến của 18.000 sinh viên, nếu 1 server chết, LB sẽ route traffic sang server khác, đảm bảo HA theo ngân sách.
- *Nếu sai:* Nếu chọn Scale up (chỉ mua server to hơn), khi server đó dính lỗi phần cứng (SPOF), hệ thống vẫn sập toàn tập, vi phạm SLA 99.9%.

Rủi ro 2: 1 Database duy nhất (SPOF & I/O Bottleneck)

- **Nó hỏng thế nào:** Hàng vạn câu lệnh SELECT lịch sự kiện và UPDATE đăng ký chỗ đập thẳng vào 1 DB, gây quá tải I/O disk, bảng bị lock, cạn kiệt connection.
- **Người dùng bị ảnh hưởng ra sao:** Hành động đăng ký quay vòng vòng (timeout), không load được danh sách workshop.
- **Cải tiến:**
 - *Chọn gì:* Tách DB thành cụm **Primary-Replica (Master-Slave)**.
 - *Vì sao:* Primary chỉ lo ghi (đăng ký chỗ), Replica chỉ lo đọc (xem lịch). Giảm tải cho DB chính.
 - *Nếu sai:* Nếu vẫn dùng 1 DB, việc tranh chấp khóa (lock contention) giữa người xem và người mua sẽ kéo sập cơ sở dữ liệu.

Rủi ro 3: Không có Cache

- **Nó hỏng thế nào:** Dữ liệu tĩnh như bản đồ, lịch sự kiện, thông tin diễn giả bị truy vấn lặp đi lặp lại từ DB cho mỗi request.
- **Người dùng bị ảnh hưởng ra sao:** Tốc độ phản hồi cực chậm.
- **Cải tiến:**
 - *Chọn gì:* Thêm một lớp **Cache (Redis/Memcached)** ở giữa App và DB.
 - *Vì sao:* Dữ liệu tĩnh sẽ được trả về trực tiếp từ RAM (<1ms), chặn 90% lượng Read traffic không cho chạm tới DB.
 - *Nếu sai:* Bỏ qua Cache sẽ khiến Read Replica chịu tải quá lớn, chi phí duy trì DB scale-up sẽ vượt ngân sách nhà trường.

Rủi ro 4: Gọi trực tiếp đồng bộ (Synchronous) sang cổng thanh toán

- **Nó hỏng thế nào:** Cổng thanh toán bên thứ 3 chậm trễ hoặc timeout. Thread trên App Server bị giữ lại (blocked) chờ phản hồi.
- **Người dùng bị ảnh hưởng ra sao:** Luồng thanh toán treo kéo theo cạn kiệt thread của toàn bộ server. Sinh viên vào xem lịch bình thường cũng không được (vi phạm nghiêm trọng yêu cầu đề bài).
- **Cải tiến:**

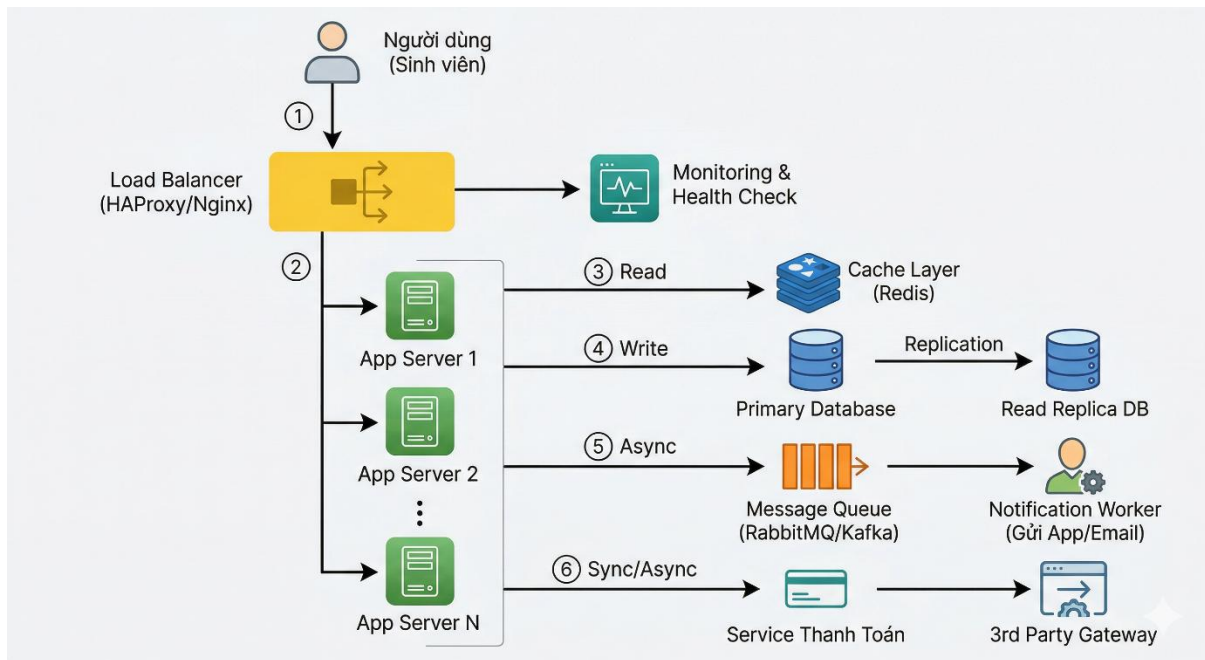
- *Chọn gì:* Tách riêng **Service Thanh toán** và dùng **Message Queue (MQ)** để xử lý bất đồng bộ, kết hợp Circuit Breaker.
- *Vì sao:* Cách ly lỗi. Thanh toán chậm chỉ ảnh hưởng luồng thanh toán, các luồng xem lịch vẫn hoạt động bình thường.
- *Nếu sai:* Nếu không tách ra, hệ thống sẽ sập dây chuyền (Cascading Failure).

Rủi ro 5: Dùng chung 1 Connection Pool cho toàn bộ chức năng

- **Nó hỏng thế nào:** Tính năng bình luận Q&A (tải liên tục) hoặc Thanh toán (giữ connection lâu) sẽ chiếm hết pool kết nối tới DB.
- **Người dùng bị ảnh hưởng ra sao:** Sinh viên chỉ muốn xem bản đồ phòng cũng bị timeout do không lấy được connection từ pool.
- **Cải tiến:**
 - *Chọn gì:* Áp dụng Bulkhead Pattern bằng cách chia nhiều **Connection Pools riêng biệt** (VD: Pool cho Read, Pool cho Write, Pool cho Thanh toán) hoặc tách thành Microservices nhỏ.
 - *Vì sao:* Cô lập tài nguyên. Chức năng Q&A bị nghẽn cũng không cướp connection của chức năng xem lịch.
 - *Nếu sai:* Không cô lập tài nguyên thì một tính năng phụ (như bình luận) có thể đánh sập tính năng chính (đăng ký workshop).

1.3. Kiến trúc Đề xuất (High-Level Architecture)

Sơ đồ khối :



Giải thích luồng chính (Main Flows) theo nghiệp vụ:

- **Luồng 1 - Xem lịch & Bản đồ (Read-Heavy):** Sinh viên -> LB -> App Server -> Kiểm tra Redis Cache. Nếu có, trả về ngay. Nếu Cache Miss, đọc từ *Read Replica DB*, cập nhật lại Cache và trả về cho sinh viên. (Đảm bảo DB Primary không bị ảnh hưởng).
- **Luồng 2 - Đăng ký chỗ (Write-Heavy, Ràng buộc cao):** App Server sử dụng *Redis* (với script *Lua* hoặc *Redis Redlock*) để đếm trừ lùi số ghế một cách atomic (giải quyết 100 ghế mà 500 người tranh nhau). Sau khi giữ chỗ thành công trên Cache, đẩy lệnh ghi xuống *Primary Database* thông qua transaction để lưu lịch sử. Đảm bảo "Không bán trùng chỗ".
- **Luồng 3 - Thanh toán (Cô lập lỗi):** Lệnh thanh toán được gửi sang *Service Thanh Toán*. Nếu gateway bên thứ 3 lỗi, *Service Thanh toán* bị nghẽn nhưng *App Server* chính không bị treo.
- **Luồng 4 - Nhận thông báo:** Sau khi thanh toán/dăng ký xong, App Server đẩy 1 event "Đăng ký thành công" vào *Message Queue*. Các *Notification Worker* sẽ từ từ tiêu thụ event này để gửi Email/App Push mà không làm chậm trải nghiệm bấm nút của sinh viên.
- **Luồng 5 - Monitoring & Health Check:** Hệ thống (như Prometheus/Grafana) liên tục ping vào các thành phần. Nếu 1 App Server dính Memory Leak, Health Check báo fail, Load Balancer lập tức đá server đó ra khỏi cụm phân tải. Đảm bảo MTTR thấp cho đội vận hành nhỏ.

4. Giải thích đánh đổi (Trade-offs)

- Chúng ta đánh đổi tính **Đơn giản của hệ thống cũ (Single Tier)** lấy tính **Phức tạp của phân tán (Distributed)**. Đội ngũ vận hành nhỏ nay phải học cách quản lý Redis, Message Queue và Replication.
- Dữ liệu lịch trình ở Cache có thể gặp tình trạng **Stale Data** (dữ liệu cũ trong vài chục giây) so với DB, nhưng đổi lại hệ thống sống sót qua 10 phút đầu (Eventual Consistency cho Read).

5. Rủi ro còn lại

- **Cache Stampede (Bão Cache):** Ngay khi mở cổng, nếu Cache trống (Cold start), hàng vạn request sẽ chọc thủng Cache và đập thẳng vào Replica DB cùng lúc.
Khắc phục: Cần viết script Pre-warm (làm nóng) Cache đưa dữ liệu lịch, số lượng ghế lên RAM trước giờ G (8:00).

Câu 2:

2.1. Nêu giả định và Phân tích ràng buộc

- **Ràng buộc nghiệp vụ:** Không được bán trùng chỗ (nghiêm cấm Overbooking).
- **Ràng buộc hiệu năng:** Lượng traffic dồn cục cực lớn (70% của 18.000 user trong 3 phút).
- **Chiến lược:** Không thể áp dụng Strong Consistency cho mọi thứ, vì thao tác đảm bảo Strong Consistency luôn đi kèm với việc **Lock (khóa)** tài nguyên, gây ra nút thắt cổ chai (bottleneck) và giảm tính sẵn sàng (Availability). Phải "hy sinh" tính nhất quán ở những nơi không ảnh hưởng đến giao dịch tài chính để cứu lấy hệ thống.

2.2. Đề xuất phương án và Giải thích đánh đổi (Áp dụng công thức 3 phần)

1. Xác nhận giữ chỗ workshop ở bước cuối cùng trước thanh toán

- **Chọn mức: Strong Consistency** (Nhất quán mạnh).
- **Vì sao:** Đây là bước chốt hạ thay đổi trạng thái hệ thống từ "còn ghế" sang "đã giữ". Phải đảm bảo tính tuần tự và cô lập (Isolation trong ACID) để tránh Race Condition khi nhiều người cùng tranh nhau 1 ghế cuối. Khi user A bấm giữ chỗ

thành công, user B bấm chậm hơn 1 mili-giây bắt buộc phải nhận được thông báo "hết chỗ".

- **Hại gì nếu chọn quá yếu (Phân tích theo yêu cầu):** Nếu chọn Eventual Consistency ở đây, hệ thống có thể cho phép cả user A và B cùng thấy còn 1 ghế và cùng vượt qua bước giữ chỗ. Hậu quả là **vi phạm ràng buộc cốt lõi của nhà trường: Bán trùng chỗ workshop (Overbooking)**, dẫn đến khủng hoảng xử lý sự cố tại hiện trường khi 2 sinh viên tranh nhau 1 cái ghế vật lý.

2. Hiện thị số ghế còn lại trên trang danh sách workshop

- **Chọn mức: Eventual Consistency** (Nhất quán cuối cùng).
- **Vì sao:** Trang danh sách là nơi chịu 90% lượng Read-traffic trong 3 phút đầu do sinh viên F5 liên tục. Việc hiển thị con số "còn 45 ghế" hay "còn 42 ghế" lệch nhau vài giây không gây chết người. Sinh viên thấy còn ghế bấm vào, nếu rủi ro hết thật thì bị chặn ở bước 1 (Xác nhận giữ chỗ).
- **Hại gì nếu chọn quá mạnh (Phân tích theo yêu cầu):** Nếu ép Strong Consistency (buộc mỗi lần F5 trang danh sách phải query đếm trực tiếp từ Primary Database để có số chính xác nhất), DB sẽ lập tức sập nguồn dưới sức ép của hàng vạn query SELECT COUNT trong 3 phút đầu. Hậu quả: **Đổi lấy con số chính xác tuyệt đối, ta trả giá bằng việc sập toàn bộ hệ thống (Mất Availability)**, sinh viên thậm chí không xem được lịch.

2.3. Cập nhật ảnh đại diện của sinh viên

- **Chọn mức: Weak Consistency** (Nhất quán yếu) hoặc **Eventual Consistency**.
- **Vì sao:** Đây là tính năng cá nhân, không mang tính giao dịch (transactional) và hoàn toàn độc lập với luồng đăng ký sự kiện. Việc sinh viên đổi avatar và phải mất vài phút (hoặc phải tải lại app) ảnh mới cập nhật trên các node Cache/CDN là hoàn toàn chấp nhận được.
- **Nếu chọn sai:** Nếu bắt buộc cập nhật avatar phải đồng bộ ngay lập tức (Strong) trên mọi server trước khi trả về 200 OK, request sẽ bị giữ (block) rất lâu một cách vô ích, lãng phí tài nguyên connection của App server.

2.4. Gửi email thông báo xác nhận đăng ký thành công

- **Chọn mức: Eventual Consistency**.
- **Vì sao:** Email là hệ thống giao tiếp với bên thứ ba (SMTP server), vốn có độ trễ cao và không ổn định. Giao dịch mua chỗ chỉ cần thành công trong DB của trường là hoàn tất. Email có thể đến sau 1 phút, hoặc 5 phút (qua Message Queue) mà không ảnh hưởng đến logic nghiệp vụ.

- **Nếu chọn sai:** Nếu gán việc gửi email vào cùng một Transaction (Strong) với việc trừ ghế, App server sẽ phải chờ SMTP server trả lời mới dám báo thành công cho sinh viên. Nếu SMTP chậm, thread bị treo, kéo theo cạn kiệt Thread Pool và làm sập luôn luồng thanh toán của người khác.

2.5. Bộ đếm số bình luận trực tiếp trong phiên Q&A

- **Chọn mức: Weak Consistency** (Nhất quán yếu).
- **Vì sao:** Trong một sự kiện livestream Q&A, comment nhảy liên tục. Việc hiển thị "1023 bình luận" hay bị rớt nhịp hiển thị "1020 bình luận" không làm thay đổi ngữ cảnh hay quyết định của bất kỳ ai. Hệ thống chỉ cần cộng dồn bất đồng bộ (fire-and-forget hoặc flush định kỳ).
- **Nếu chọn sai:** Việc áp Strong Consistency cho bộ đếm này (cứ 1 comment là lock row trong DB để UPDATE count = count + 1) sẽ tạo ra tình trạng khóa chết (Deadlock) ở DB khi có bão bình luận, làm tính năng Q&A bị tê liệt hoàn toàn.

2.6. Dashboard thống kê cho ban tổ chức

- **Chọn mức: Eventual Consistency.**
- **Vì sao:** Ban tổ chức không cần nhìn thấy biểu đồ đăng ký nhảy múa theo từng mili-giây. Họ chỉ cần dữ liệu cập nhật theo batch (ví dụ: quét từ Read Replica mỗi 1-5 phút một lần). Việc này giúp tách biệt (decouple) luồng phân tích dữ liệu (OLAP) ra khỏi luồng giao dịch của sinh viên (OLTP).
- **Nếu chọn sai:** Nếu dùng Strong Consistency (buộc Dashboard phải đọc trực tiếp từ Primary Database đang bận rộn xử lý đăng ký), các câu lệnh gom nhóm (GROUP BY, SUM, COUNT) nặng nề của Dashboard sẽ chiếm dụng I/O và CPU của DB. Hậu quả: Sinh viên đang đăng ký sẽ bị timeout do DB bận phục vụ việc vẽ biểu đồ cho ban tổ chức.

Câu 3:

3.1. Nêu giả định và Phân tích ràng buộc

- **Giả định:** Do 70% truy cập (khoảng 12.600 user) dồn vào 3 phút đầu, hệ thống sẽ trải qua một cú sốc Spike Traffic rất mạnh. Sau 10 phút, lượng truy cập sẽ giảm sâu và ổn định trở lại.
- **Ràng buộc:** Ngân sách có hạn, không thể mua dư thừa tài nguyên (Over-provisioning) chạy 24/7 chỉ để phục vụ 10 phút đầu của một sự kiện. Phải dùng chiến lược mở rộng linh hoạt.

3.2. Đề xuất phương án mở rộng cho từng tầng

1. Web/App Layer

- **Chọn gì:** Ưu tiên **Horizontal Scaling (Mở rộng ngang)** bằng cách đặt nhiều instance nhỏ sau một Load Balancer, kết hợp Auto-scaling group.
- **Vì sao:** Tầng App/Web thường thiết kế phi trạng thái (Stateless). Trong 3 phút đầu, ta có thể tự động "spin up" (nhân bản) từ 2 lên 10 server để hứng bão traffic, sau 10 phút thì tắt đi để tiết kiệm ngân sách.
- **Tác động:**
 - *Latency:* Tăng nhẹ (vài mili-giây) do tốn thêm một nhịp định tuyến qua Load Balancer.
 - *Throughput:* Tăng đột phá, hệ thống có thể xử lý hàng chục ngàn Request/giây (RPS) đồng thời.
- **Nút thắt cổ chai dễ gặp:** Nếu code viết không tối ưu (ví dụ: cấp phát bộ nhớ bừa bãi gây rò rỉ), việc mở rộng ngang chỉ làm tăng số lượng server bị... Crash (OOM - Out of Memory) chứ không giải quyết được vấn đề gốc rễ.

2. Database Layer

- **Chọn gì:** Kết hợp **Vertical Scaling (Mở rộng dọc - tăng RAM/CPU)** cho Primary DB và **Horizontal Scaling (Mở rộng ngang)** bằng cách thêm các Read Replicas.
- **Vì sao:** Bài toán yêu cầu *Không bán trùng chỗ*. Xử lý transaction chống Overbooking rất phức tạp nếu dùng DB phân tán (Sharding). Do đó, để an toàn và vừa ngân sách, ta tối đa sức mạnh (Vertical) cho 1 node Primary chỉ để lo việc Write (giữ chỗ). Phần Read (xem lịch - chiếm 90% traffic) sẽ được đẩy sang cụm Read Replica được mở rộng ngang.
- **Tác động:**
 - *Latency:* Read latency giảm mạnh do chia sẻ tải. Write latency có thể bị nghẽn nhẹ nếu Primary DB phải khóa (Lock) các dòng dữ liệu (row-level lock) khi hàng trăm người tranh nhau 1 workshop 50 chỗ.
 - *Throughput:* Read throughput tăng tuyến tính theo số lượng Replica.
- **Nút thắt cổ chai dễ gặp:** Tranh chấp khóa (Lock contention) tại Primary DB. Dù CPU/RAM mạnh đến đâu, nếu 500 người cùng UPDATE vào 1 dòng số_ghế của workshop A, DB vẫn phải xếp hàng xử lý tuần tự.

3. Cache Layer

- **Chọn gì:** Ưu tiên **Horizontal Scaling** (Ví dụ: Redis Cluster phân mảnh dữ liệu).
- **Vì sao:** Nếu dùng 1 node Redis cực mạnh (Vertical), ta vẫn sẽ chết vì Redis xử lý đơn luồng (Single-threaded) và giới hạn số lượng connection. Chia nhỏ dữ liệu ra nhiều node giúp tận dụng nhiều CPU core từ nhiều máy.
- **Tác động:**
 - *Latency:* Rất thấp (luôn ở mức $< 1\text{ms}$).
 - *Throughput:* Cực kỳ cao, gánh toàn bộ sức ép đọc từ DB.
- **Nút thắt cổ chai dễ gặp:** Vấn đề **Hot Key**. Sẽ có một workshop "hot" nhất (vd: Workshop về AI) mà cả 18.000 sinh viên cùng xem. Dù có Redis Cluster, toàn bộ request cho cái workshop đó vẫn đập chung vào 1 node cụ thể chứa khóa dữ liệu đó, làm node này quá tải trong khi các node khác ngồi chơi.

4. Notification Layer

- **Chọn gì:** Ưu tiên **Horizontal Scaling** bằng cách tạo ra nhiều Worker Nodes đứng sau một Message Queue (RabbitMQ/Kafka).
- **Vì sao:** Gửi email hay push app phụ thuộc vào mạng và dịch vụ ngoài, rất tốn thời gian. Cứ thêm nhiều Worker (Consumer) thì việc "làm thịt" đồng hàng đợi càng nhanh.
- **Tác động:**
 - *Latency:* Latency tổng thể để nhận được email khá cao (vài giây đến vài phút), nhưng Latency đối với user trải nghiệm (bấm nút đăng ký) lại cực thấp vì hệ thống chỉ bắn event vào Queue là trả kết quả ngay (Asynchronous).
 - *Throughput:* Tăng mạnh khả năng xử lý thông báo song song.
- **Nút thắt cổ chai dễ gặp:** Bị "bóp băng thông" (Rate Limiting) từ nhà cung cấp dịch vụ Email thứ 3 (ví dụ: SendGrid/Amazon SES chỉ cho gửi tối đa 100 email/giây). Worker lấy việc ra nhanh nhưng dịch vụ đầu cuối từ chối phục vụ.

3.3. Giải quyết yêu cầu bắt buộc: Latency thấp chưa chắc đồng nghĩa với Throughput cao

Trường hợp chứng minh tại UniEvent: Giả sử hệ thống gọi sang **Cổng thanh toán bên thứ ba**.

- Ngân hàng đối tác đầu tư server cực xịn, thời gian xử lý mỗi giao dịch (Latency) cực kỳ thấp: **Chỉ mất 20ms**.

- Tuy nhiên, phía App Server của trường (do cấu hình yếu) chỉ được thiết lập cấu hình **Connection Pool là 10 connections** để kết nối tới Gateway thanh toán.

Hậu quả: Mỗi kết nối giải quyết xong 1 request mất 20ms. Vậy 1 kết nối trong 1 giây xử lý được $1000\text{ms} / 20\text{ms} = 50$ requests. Với 10 kết nối, hệ thống chỉ có thể xử lý tối đa $10 * 50 = 500$ requests / giây. -> **Throughput bị "đóng trần" ở mức 500 RPS.**

Khi 12.600 sinh viên ập vào thanh toán cùng lúc, dù thời gian xử lý lỗi (Latency) rất nhanh, nhưng thông lượng (Throughput) hệ thống không đủ sức chứa. Hàng vạn sinh viên phải chờ đợi trong hàng đợi của App server để "tranh giành" 10 connection ít ỏi đó, dẫn đến hiện tượng xếp hàng kéo dài (Queueing delay) và cuối cùng là Timeout sập hệ thống.

=> **Kết luận:** Để hệ thống chịu tải tốt, Latency thấp là điều kiện cần (chạy nhanh), nhưng Throughput cao (đủ đường truyền/connection/thread để chạy song song) mới là điều kiện đủ.

3.4. Rủi ro còn lại (Của thiết kế mở rộng)

Việc sử dụng Auto-scaling (Web layer) thường có độ trễ. Quá trình nhận diện tải cao -> khởi tạo server mới -> cài môi trường -> đưa vào Load balancer thường mất từ 1 đến 3 phút. Trong khi đó, bão traffic của 18.000 sinh viên đổ ập tới ngay trong phút đầu tiên. Nếu hệ thống tự động scale quá chậm (Reactive scaling), nó sẽ sập trước khi kịp có thêm viện binh. *Đề xuất vận hành bổ sung:* Phải cấu hình **Pre-warming** (Scale up thủ công hệ thống lên kích thước tối đa vào lúc 7:45 sáng, trước khi mở cổng, sau đó cấu hình tự động thu nhỏ (Scale-in) vào lúc 8:30 để tiết kiệm tiền).

CÂU 4:

4.1 Nêu giả định và Phân tích ràng buộc

- **Giả định:** Hệ thống sử dụng cơ chế replication đa điểm (Leaderless hoặc Multi-leader) để đảm bảo dữ liệu luôn có bản sao trên 5 máy chủ vật lý khác nhau.
- **Ràng buộc nghiệp vụ:**
 - Luồng xem (Read) chiếm số lượng cực lớn, cần tốc độ phản hồi nhanh để sinh viên không bị ức chế.
 - Luồng đặt chỗ (Write) ít hơn nhưng yêu cầu độ chính xác tuyệt đối để không vi phạm ràng buộc "không bán trùng chỗ" của nhà trường.

- Ngân sách chỉ đủ cho High Availability (HA), do đó khi có sự cố mạng (Partition), ta phải chọn ưu tiên giữa việc "cho xem tiếp nhưng có thể sai số" hay "dừng lại để đảm bảo đúng".

4.2. Đề xuất phương án cho từng tình huống

Tình huống a: Người dùng mở trang xem workshop còn bao nhiêu chỗ (Luồng Đọc)

- **Chọn Quorum: $W=3, R=1$.**
- **Giải thích vì sao:** * Với $R=1$, App Server chỉ cần hỏi đúng 1 node gần nhất là có thể trả về số lượng ghế cho sinh viên. Điều này cực kỳ quan trọng trong 3 phút đầu khi có 12.600 người cùng F5 trang web. Tốc độ (Latency) là ưu tiên hàng đầu ở đây.
 - Giá trị $W=3$ (Majority) đảm bảo khi có lệnh ghi, dữ liệu đã được lưu an toàn trên đa số các node.
- **Nghiên về CP hay AP: Nghiên về AP (Availability & Partition Tolerance).**
 - Khi xảy ra chia cắt mạng, sinh viên vẫn có thể xem được danh sách workshop từ bất kỳ node nào còn sống. Hệ thống ưu tiên việc "phải phản hồi" hơn là việc "phải đúng 100% từng giây".
- **Rủi ro cụ thể nếu chọn sai (Ví dụ chọn $R=3$):** * Nếu chọn $R=3$ cho luồng xem, hệ thống sẽ trở nên cực kỳ chậm chạp vì mỗi lần sinh viên load trang, App Server phải chờ phản hồi từ ít nhất 3 node.
 - **Hậu quả:** Trong giờ cao điểm, latency tăng cao sẽ gây ra hiệu ứng hàng đợi (Queueing delay), làm sập App server trước cả khi DB bị quá tải. Hệ thống mất tính sẵn sàng (Availability), vi phạm SLA 99.9%.

Tình huống b: Người dùng bấm xác nhận đăng ký và sắp thanh toán (Luồng Ghi)

- **Chọn Quorum: $W=3, R=3$.** (Thỏa mãn $W + R > N$, tức $3 + 3 > 5$).
- **Giải thích vì sao:** * Đây là cấu hình đảm bảo **Strong Consistency (Nhất quán mạnh)**. Khi $W+R > N$, tập hợp các node được ghi và tập hợp các node được đọc chắc chắn sẽ có ít nhất một node giao thoa (overlap). Node giao thoa này đảm bảo user sau luôn đọc được thông tin mới nhất mà user trước vừa ghi (ví dụ: ghế đã bị lấy mất).
 - Điều này giúp hệ thống kiểm tra chính xác workshop còn ghế hay không trước khi tiến hành thanh toán.
- **Nghiên về CP hay AP: Nghiên về CP (Consistency & Partition Tolerance).**

- Theo yêu cầu của nhà trường "Không được bán trùng chỗ", hệ thống bắt buộc phải nhất quán. Nếu mạng bị chia cắt và không đủ 3 node để đạt quorum, hệ thống thà từ chối giao dịch (ngừng cho đăng ký) còn hơn là cho phép đăng ký bừa bãi dẫn đến sai lệch dữ liệu.
- **Rủi ro cụ thể nếu chọn sai (Ví dụ chọn $W=2, R=2$):** * Lúc này $W + R = 4$, nhỏ hơn $N=5$. Có khả năng user A ghi việc giữ chỗ vào node 1 và 2, nhưng user B lại đọc số ghế từ node 3 và 4 (vốn chưa kịp cập nhật).
 - **Hậu quả:** Cả hai user đều thấy còn 1 ghế cuối và cùng bấm đăng ký thành công. Hệ thống rơi vào tình trạng **Overbooking (Bán trùng chỗ)**, vi phạm ràng buộc nghiệp vụ nghiêm trọng nhất của đề bài.

4.3. Giải thích đánh đổi (Trade-offs)

- Ở tình huống (a), ta chấp nhận đánh đổi **Consistency lấy Availability**. Sinh viên có thể thấy số ghế là 50 nhưng thực tế có thể là 48 (Stale read). Tuy nhiên, điều này chấp nhận được vì người dùng sẽ được kiểm tra lại ở bước (b).
- Ở tình huống (b), ta đánh đổi **Availability lấy Consistency**. Nếu cụm DB bị chết 3 trên 5 node, hệ thống sẽ ngừng cho phép đăng ký mới. Đây là cái giá phải trả để bảo vệ tính toàn vẹn của dữ liệu số ghế.

4.4. Rủi ro còn lại

Mặc dù $W=3, R=3$ đảm bảo nhất quán ở mức lưu trữ, nhưng trong các hệ thống phân tán vẫn có rủi ro về **Clock Skew (Lệch đồng hồ)** giữa các node. Nếu các node dựa vào timestamp để phân định ai đăng ký trước, việc lệch giờ vài mili-giây có thể khiến dữ liệu bị ghi đè sai lệch (Last-write-wins).

Khắc phục: Cần sử dụng các cơ chế như *Vector Clocks* hoặc *Lamarkian timestamps* để định danh thứ tự thao tác chính xác thay vì chỉ dùng giờ hệ thống.

Câu 5:

5.1. Nêu giả định và Phân tích ràng buộc

- **Giả định:** Thời điểm 8:05 sáng, đã qua "cú sốc" 3 phút đầu nhưng lượng traffic vẫn rất cao. Hệ thống đang phải gồng gánh nhiều tính năng cùng lúc (Đọc, Ghi, Thanh toán, Streaming Comment).

- **Ràng buộc:** Nhà trường yêu cầu lỗi thanh toán không được làm sập chức năng xem lịch. Đội vận hành nhỏ, do đó các cơ chế bảo vệ phải tự động (Circuit Breaker, Bulkhead) và việc can thiệp bằng tay phải cực kỳ gọn.
-

5.2. Đề xuất phương án và Giải thích

a) Retry với Exponential Backoff (Cổng thanh toán lỗi 503)

- **Chọn gì:**
 - **Số lần tối đa:** 3 lần.
 - **Khoảng chờ:** Tăng theo cấp số nhân (ví dụ: 2s, 4s, 8s) và **bắt buộc cộng thêm Jitter** (thời gian ngẫu nhiên, ví dụ: $2s + \text{random}(0.1, 0.5)s$) để tránh hiện tượng "Thundering Herd" (nhiều request cùng retry vào đúng một tích tắc gây sập tiếp).
 - **Khi nào dừng:** Dừng ngay lập tức nếu hết 3 lần, HOẶC khi Circuit Breaker đã báo trạng thái OPEN.
- **Vì sao:** Lỗi 503 (Service Unavailable) cho thấy đối tác đang quá tải. Việc chờ lâu hơn sau mỗi lần thử giúp họ có thời gian "thở" để phục hồi.
- **Hậu quả nếu sai:** Nếu retry vô hạn hoặc không có thời gian chờ (backoff), App Server của ta sẽ dội bom (DDoS) ngược lại cổng thanh toán, đồng thời vắt kiệt Thread Pool của chính mình vì các luồng cứ phải đứng đợi.

b) Kích hoạt Circuit Breaker (Cắt mạch)

- **Chọn gì:**
 - **Dấu hiệu mở mạch (Open):** Tỷ lệ lỗi (lỗi 5xx hoặc timeout) vượt quá ngưỡng 50% trong 10 giây qua, hoặc có 20 request thất bại liên tiếp đến cổng thanh toán.
 - **Chuyển sang Half-open (Nửa mở):** Sau một thời gian làm mát (Cooldown period) khoảng 30 - 60 giây. Lúc này hệ thống chỉ cho lọt 1-2 request đi qua để "thăm dò". Nếu 2 request này thành công, đóng mạch (Closed - bình thường lại). Nếu thất bại, mở mạch tiếp 60 giây nữa.
- **Vì sao:** Nguyên tắc Fail-fast. Khi biết chắc đối tác đã sập, không cần gọi nữa. Việc mở mạch giúp trả lỗi ngay lập tức (Fast Failure) để giải phóng connection/thread cho hệ thống, bảo vệ App Server.
- **Hậu quả nếu sai:** Không có Circuit Breaker, hệ thống sẽ gặp "Cascading Failure" (Sập dây chuyền). Thanh toán sập kéo theo App Server sập, App Server sập kéo theo cả web xem lịch bị sập (vi phạm nghiêm trọng yêu cầu nhà trường).

c) Áp dụng Bulkhead Pattern (Vách ngăn)

- **Chọn gì:** Áp dụng ở mức **Thread Pool / Connection Pool** của App Server. Ta chia nhỏ tài nguyên: Ví dụ 60% luồng cho Xem lịch, 30% luồng cho Đăng ký/Thanh toán, và chỉ cấp đúng 10% luồng cho Dịch vụ Bình luận Q&A.
- **Vì sao:** Để cô lập thảm họa. Hiện tại dịch vụ bình luận tăng đột biến, nó sẽ ngốn hết 10% tài nguyên của nó và bị nghẽn (người bình luận sẽ thấy chậm/lỗi). Nhưng 90% tài nguyên còn lại vẫn an toàn để phục vụ sinh viên xem lịch và giữ chỗ.
- **Hậu quả nếu sai:** Nếu dùng chung một Pool (như kiến trúc ban đầu), chức năng Q&A vô thường vô phạt sẽ chiếm trọn 100% thread của server, khiến sinh viên không thể load được lịch sự kiện.

d) Graceful Degradation (Suy thoái có kiểm soát)

- **Chọn gì:**
 - **Tính năng giữ bằng mọi giá:** Xem lịch, Xem bản đồ, và Bấm nút giữ chỗ (trừ số lượng ghế thành công trên hệ thống).
 - **Tính năng tắt tạm / suy thoái:**
 - *Với Q&A:* Chuyển sang chế độ "Read-only" (ẩn khung nhập, chỉ cho xem bình luận cũ) để cứu CPU/Thread.
 - *Với Thanh toán (Đang lỗi 503):* Cho phép người dùng vượt qua bước này, chuyển trạng thái vé thành "**Pending Payment**" (Chờ thanh toán). Thông báo: "Chỗ của bạn đã được giữ. Hệ thống thanh toán đang bảo trì, vui lòng thanh toán trong 24h tới".
- **Vì sao:** Cốt lõi của "Tuần lễ định hướng" là sinh viên có được suất dự workshop. Tiền có thể thu sau, nhưng nếu không cho họ giữ chỗ, họ sẽ hoảng loạn và F5 liên tục.
- **Hậu quả nếu sai:** Thiết kế cứng nhắc "không thanh toán được thì không cho đăng ký" sẽ gây phẫn nộ cho user, tạo ra lượng truy cập ảo khổng lồ khi hàng vạn người cứ canh me tải lại trang.

e) Thứ tự xử lý ưu tiên của kỹ sư trực ca (10 phút đầu)

Nguyên tắc SRE: "Cứu người dùng đang sống trước, chữa bệnh cho server sau".

1. **Ưu tiên 1 (Phút 1-2): Cô lập App Server bị memory leak.** Truy cập vào Load Balancer, "drain" (rút hết traffic) hoặc "remove" con server chậm chạp đó ra khỏi cụm. Để cho hệ thống Auto-scaling tự động bật một server mới thay thế. Trải nghiệm của user đi vào server đó sẽ hết bị lag lập tức.

2. **Ưu tiên 2 (Phút 3-5): Bật chiến lược Degradation.** Kiểm tra xem Circuit Breaker của thanh toán đã nhảy chưa, chủ động gạt "công tắc" kích hoạt cơ chế "Pending Payment" cho toàn trường và tắt chức năng bình luận Q&A.
 3. **Ưu tiên 3 (Phút 6-10): Kiểm tra Read Replica.** DB đọc mất kết nối tạm thời thường do mạng chập chờn. Kiểm tra xem Load Balancer của DB đã tự động loại node đó ra và đẩy traffic sang các Replica khác hoặc Cache chưa. Nếu Cache vẫn đang gánh tốt (như thiết kế ở Câu 1), thì bỏ qua, từ từ fix sau 10 phút cao điểm.
-

5.3. Rủi ro còn lại

Việc sử dụng Graceful Degradation (Cho phép "Pending Payment") sẽ dời áp lực thanh toán về tương lai. Rủi ro là vào ngày hôm sau, khi có thông báo "Yêu cầu thanh toán để giữ chỗ", sinh viên lại đồng loạt ập vào, tạo ra một đợt bão traffic mới.

Cách khắc phục: Khi hệ thống thanh toán sống lại, cần gửi email thông báo trả tiền theo từng đợt (batch) cho từng nhóm 1000 sinh viên, giãn cách nhau 15 phút.

Câu 6:

6.1. Nêu giả định

- **Tổng thời gian hoạt động dự kiến trong 1 tháng (30 ngày):** $30 \text{ ngày} * 24 \text{ giờ} * 60 \text{ phút} = 43.200 \text{ phút}$.
 - Mọi sự cố đều làm hệ thống ngừng hoạt động hoàn toàn (Downtime toàn phần).
-

6.2. Tính toán các chỉ số hệ thống (Từ a đến e)

a) Tổng downtime trong tháng

- Sự cố 1 (45 phút) + Sự cố 2 (20 phút) + Sự cố 3 (120 phút) = **185 phút**.

b) MTTR (Mean Time To Recovery - Thời gian phục hồi trung bình)

- $MTTR = \text{Tổng downtime} / \text{Số lượng sự cố}$
- $MTTR = 185 \text{ phút} / 3 \text{ sự cố} = \mathbf{61,67 \text{ phút/sự cố}}$.

c) Uptime % trong tháng (Tỷ lệ khả dụng)

- Uptime = Tổng thời gian dự kiến - Tổng downtime
- Uptime = 43.200 - 185 = 43.015 phút.
- Tỷ lệ Uptime % = $(43.015 / 43.200) * 100 = 99,57\%$.

d) Hệ thống có đạt SLA 99.9% không?

- **Không.**
- *Giải thích:* Để đạt SLA 99.9%, mức downtime tối đa cho phép trong 1 tháng là: $43.200 * (1 - 0.999) = 43,2 \text{ phút}$. Thực tế hệ thống đã sập tới 185 phút, vượt xa giới hạn cho phép. (Nhà trường có quyền phạt đơn vị phát triển).

e) Hệ thống có đạt SLO 99.95% không?

- **Không.**
- *Giải thích:* SLO là cam kết nội bộ khắt khe hơn. Giới hạn downtime cho 99.95% chỉ là: $43.200 * (1 - 0.9995) = 21,6 \text{ phút}$. Uptime thực tế (99,57%) thấp hơn rất nhiều so với mục tiêu (99,95%).

6.3. Đề xuất biện pháp giảm MTTR (f)

Để giảm MTTR (Thời gian từ lúc hệ thống sập đến lúc được chữa xong) cho tháng sau, mình đề xuất một biện pháp kỹ thuật kết hợp vận hành theo đúng công thức "3 phần":

- **Chọn gì:** Triển khai cơ chế **Automated Health Check** kết hợp **Auto-Failover** tại tầng Load Balancer, kết hợp viết **Runbook (Kịch bản xử lý sự cố chuẩn hóa)** cho đội trực ca.
- **Vì sao (Gắn với nghiệp vụ):** MTTR bao gồm 2 pha: Thời gian phát hiện (Detect) và Thời gian sửa chữa (Fix).
 - Với *Health Check tự động*, khi 1 App Server bị treo, Load Balancer lập tức phát hiện sau 5 giây và loại nó ra khỏi cụm (Failover), giúp giảm thời gian chẩn đoán xuống gần như bằng 0.
 - Với *Runbook*, khi sự cố lớn như "Cổng thanh toán sập" xảy ra lúc rạng sáng, kỹ sư (dù là người mới) chỉ cần mở tài liệu, copy-paste các lệnh/ấn nút kích hoạt Circuit Breaker đã được định nghĩa sẵn, thay vì phải mò mẫm hộp bàn tìm nguyên nhân. Điều này giúp rút ngắn thời gian sửa chữa từ "hàng giờ" (như sự cố 120 phút trong đề) xuống còn "vài phút".
- **Hậu quả nếu chọn sai:** Nếu giải pháp chỉ là "cử thêm người trực màn hình 24/7" (cách vận hành thủ công), con người luôn có độ trễ phản xạ, dễ buồn ngủ và dễ hoảng loạn khi báo log đổ về. Hệ thống không những không phục hồi nhanh mà

kỹ sư còn có nguy cơ gõ sai lệnh (Human Error), làm hệ thống hỏng nặng hơn, đẩy MTTR lên mức cao kỷ lục.

Câu 7:

Lựa chọn phản biện: > **"Cho toàn bộ chức năng dùng chung một thread pool/connection pool cho dễ quản lý."**

7.1. Vì sao phương án này hấp dẫn lúc đầu? (Cái bẫy ngọt ngào)

- **Dễ lập trình, dễ cấu hình:** Lập trình viên không cần phải đau đầu chia cắt kiến trúc hay cấu hình nhiều data source phức tạp. Chỉ cần khai báo 1 file application.properties với max-pool-size = 200 là xong. Toàn bộ tính năng từ xem lịch, đăng ký, đến bình luận đều "múc" chung từ cái giếng này.
- **Tối ưu tài nguyên (trên lý thuyết):** Khi hệ thống chạy ở trạng thái bình thường (tải thấp), các chức năng có thể chia sẻ tài nguyên cho nhau một cách linh hoạt, không bị tình trạng "pool này rảnh rỗi trong khi pool kia thiếu hụt". Quản trị viên chỉ cần monitor 1 chỉ số pool duy nhất.

7.2. Vì sao nó sai và cực kỳ nguy hiểm trong bối cảnh UniEvent?

Hệ thống UniEvent có đặc thù là **sự chênh lệch cực lớn về tốc độ xử lý (latency)** và **tầm quan trọng** giữa các tính năng:

- **Tính năng xem lịch:** Nhanh (chỉ truy vấn DB hoặc Cache mất < 5ms) và sống còn.
- **Tính năng Q&A / Thanh toán:** Chậm (giữ connection lâu do chờ mạng hoặc I/O) và phụ.

Nếu dùng chung một pool, ta đã vi phạm nguyên tắc **Cô lập lỗi (Fault Isolation)**. Hệ thống lúc này giống như một con tàu không có các khoang chống ngập rò rỉ (Bulkhead). Nước tràn vào một phòng sẽ làm chìm cả con tàu. Khi một tính năng chậm chạp (như cổng thanh toán bị nghẽn) hoặc đột biến (bão comment Q&A), nó sẽ "ăn vạ" và giữ rịt lấy toàn bộ thread/connection trong pool mà không chịu nhả ra.

7.3. Hậu quả cụ thể có thể xảy ra (Cascading Failure - Sập dây chuyền)

Hãy tưởng tượng kịch bản lúc 8:05 sáng (giống Câu 5):

1. Cổng thanh toán bên thứ ba bị lỗi, mất 30 giây mới trả về báo lỗi timeout.
2. Hàng ngàn sinh viên bấm "Thanh toán". Lập tức, 200 connection trong pool dùng chung bị hệ thống giao cho luồng thanh toán.

3. Cả 200 connection này bị "treo" cứng (block) trong suốt 30 giây để chờ cổng thanh toán phản hồi.
4. Ngay lúc đó, một sinh viên khác vào App chỉ với mục đích rất đơn giản: **Xem bản đồ phòng (đáng lý chỉ mất 1 mili-giây tải từ Cache)**.
5. Tuy nhiên, App Server không còn lấy ra được thread/connection nào trống từ pool chung nữa. Request của sinh viên này bị đưa vào hàng đợi, quá thời gian (timeout) và nhận về lỗi 503 Service Unavailable hoặc màn hình trắng xóa.

Hậu quả cuối cùng: Một dịch vụ bên thứ ba (thanh toán) bị lỗi đã lôi theo một tính năng phụ (Q&A), rồi lôi luôn cả tính năng cốt lõi (Xem lịch, Bản đồ) chết chum theo. Vi phạm hoàn toàn cam kết ràng buộc của đề bài: *"Nếu cổng thanh toán gặp sự cố, sinh viên vẫn phải xem được lịch sự kiện"*.

7.4. Chốt lại vấn đề (Khẳng định kiến trúc đúng)

Ngôn ngữ kỹ thuật gọi hiện tượng này là rủi ro **"Noisy Neighbor" (Hàng xóm ồn ào)** kết hợp **"Resource Exhaustion" (Cạn kiệt tài nguyên)**.

Thay vì dùng chung, thiết kế chuẩn bắt buộc phải áp dụng **Bulkhead Pattern**: Chia nhỏ tài nguyên thành các pool độc lập. Ví dụ: Pool A (100 threads) chỉ dành cho Đọc/Xem lịch; Pool B (50 threads) dành cho Thanh toán. Lúc đó, cổng thanh toán có sập thì cũng chỉ cháy Pool B, Pool A vẫn khỏe mạnh phục vụ sinh viên xem lịch bình thường, cứu sống SLA của hệ thống!